



Universidad Don Bosco, El Salvador

**DESARROLLO DE SOFTWARE PARA MÓVILES
DSM 941 G01T**

Trabajo de investigación

Docente:

Alexander Alberto Siguenza Campos

Estudiantes

Nombres	Apellidos	Carnet
Carlos David	Guevara Martínez	GM172474
Fernando Josue	Torres Lara	TL222729
Cristian Omar	Pineda Campos	PC222718
Geovany Arturo	Pineda Fuentes	PF211251

Grupo 9

Investigar y desarrollar una aplicación en Android con login y sistema de ingreso de notas usando Jetpack Compose.

Índice

Introducción	2
Jetpack Compose	3
Ventajas de Jetpack Compose	4
Descripción del flujo implementado y decisiones de diseño	5
Descripción del flujo implementado	5
Punto de Entrada (MainActivity)	5
Gestión de Sesión y Autenticación	5
Gestión de Materias (SubjectsScreen)	6
Gestión de Actividades y Notas (SubjectDetailScreen)	6
Cálculo en tiempo real	6
Decisiones de diseño	7
UI Declarativa (Jetpack Compose)	7
Gestión de Estado Local	7
SQLite Nativo (Sin ORM)	7
Patrón Repositorio	7
Conclusiones	8
Bibliografía	9
Evidencias	10

Introducción

El desarrollo de aplicaciones móviles ha experimentado una transición significativa hacia paradigmas declarativos que optimizan la construcción de interfaces de usuario. En el ecosistema de Android, Jetpack Compose ha surgido como el toolkit nativo y moderno, impulsado por Kotlin, diseñado para reemplazar el sistema tradicional de diseño basado en vistas. Este enfoque permite construir interfaces más eficientes y reactivas, destacando por su capacidad para reducir el código repetitivo, facilitar una integración fluida con la lógica de la aplicación y mejorar sustancialmente el manejo del estado. Comprender y aplicar estas ventajas tecnológicas es esencial en las prácticas actuales de ingeniería de software para dispositivos móviles. Con base en lo anterior, el presente documento tiene como propósito analizar la implementación práctica de una interfaz de usuario utilizando esta tecnología. En primer lugar, se describe detalladamente el flujo de interacción desarrollado y la arquitectura subyacente del proyecto.

Jetpack Compose

Jetpack Compose se presenta como una herramienta de vanguardia concebida para optimizar la creación de interfaces de usuario (UI). Esta tecnología fusiona el paradigma de la programación reactiva con la sintaxis concisa y eficiente del lenguaje Kotlin. Su arquitectura es inherentemente declarativa; es decir, en lugar de construir la interfaz paso a paso de forma imperativa, el desarrollador la define invocando funciones que traducen directamente los datos en una jerarquía visual. Asimismo, el entorno de trabajo (*framework*) está diseñado para detectar cualquier modificación en los datos subyacentes, lo que desencadena una actualización automática de la interfaz sin requerir intervención manual adicional.

El núcleo arquitectónico de una aplicación desarrollada bajo este modelo se basa en funciones de componibilidad, comúnmente denominadas *composables*. Estas consisten en métodos estándar de programación que incorporan la anotación `@Composable`, un identificador que habilita al sistema para gestionar, rastrear y refrescar la interfaz a lo largo del ciclo de vida de la aplicación. Un aspecto destacable de este enfoque es que permite invocar otras funciones de la misma naturaleza, promoviendo la segmentación del código en unidades lógicas y funcionales más reducidas.

Esta metodología orientada a la creación de componentes pequeños y reutilizables facilita la consolidación de una biblioteca de elementos gráficos estandarizados dentro de la aplicación. Dado que cada elemento componible asume la responsabilidad exclusiva de renderizar una sección específica de la pantalla, es posible modificar y escalar cada parte del diseño de manera independiente, lo que mejora sustancialmente el mantenimiento y la organización del proyecto.

Ventajas de Jetpack Compose

La adopción de Jetpack Compose ofrece diversas ventajas técnicas que optimizan el ciclo de desarrollo de aplicaciones móviles. Entre estas se encuentran las siguientes:

- **Reducción de código y mayor legibilidad:** La eliminación de los tradicionales archivos XML y la definición de la interfaz visual íntegramente mediante funciones componibles en Kotlin disminuyen las líneas de programación necesarias y simplifican de forma considerable el mantenimiento a largo plazo del proyecto.
- **Interfaz de usuario reactiva y dinámica:** La integración nativa con herramientas de gestión de estado y asincronía, como State y Flow, permite que la interfaz gráfica se actualice automáticamente ante cualquier modificación en los datos subyacentes. Esto elimina la complejidad de las actualizaciones manuales de estado y reduce la probabilidad de introducir errores lógicos.
- **Compatibilidad integral con Material Design 3:** El framework incorpora bibliotecas nativas y actualizadas que facilitan la implementación de temas dinámicos. Esto permite adaptar la estética de la aplicación a las tendencias contemporáneas de diseño centrado en la personalización de manera ágil e intuitiva.
- **Rendimiento optimizado y herramientas avanzadas:** El ecosistema de Android Studio proporciona instrumentos de desarrollo sumamente maduros, tales como la previsualización en tiempo real, el seguimiento detallado de recomposiciones (recomposition

tracking), métricas de rendimiento y soporte robusto para animaciones complejas, agilizando así el flujo de trabajo en la ingeniería de software.

Descripción del flujo implementado y decisiones de diseño

Descripción del flujo implementado

La aplicación sigue un flujo lineal y lógico enfocado en la gestión de materias y sus respectivas evaluaciones.

Punto de Entrada (MainActivity)

La aplicación inicia estableciendo el tema principal de diseño (RegistroNotasAppTheme) y carga el grafo de navegación (AppNavGraph).

Gestión de Sesión y Autenticación

- Al abrir la app, el sistema evalúa a través de AppNavGraph y las rutas (AppRoutes) si el usuario debe ir a la pantalla principal (MainScreen), la cual actúa como un landing page ofreciendo las opciones de Iniciar Sesión o Registrarse.
- La persistencia de la sesión se maneja dinámicamente con la clase SessionManager. Al iniciar sesión exitosamente, se guarda el ID y nombre del usuario.

Gestión de Materias (SubjectsScreen)

- Una vez autenticado, el usuario es redirigido a la vista de materias. Aquí, la aplicación carga las materias asociadas específicamente a su userId usando el SubjectRepository.
- El usuario puede agregar una nueva materia (con validaciones de campos vacíos y nombres duplicados), editar el nombre de una materia existente o eliminarla. Al eliminar una materia, se eliminan en cascada todas sus actividades asociadas.
- Se provee la opción de cerrar sesión, la cual limpia el SessionManager y devuelve al usuario a la pantalla inicial.

Gestión de Actividades y Notas (SubjectDetailScreen)

- Al tocar una materia, se navega al detalle de la misma pasando el subjectId como parámetro.
- En esta pantalla se listan las actividades (tareas, exámenes, etc.). Por cada actividad, el usuario define un nombre, el porcentaje de evaluación (ej. 25%) y la nota obtenida (de 0 a 10).

Cálculo en tiempo real

- A medida que se agregan, editan o eliminan actividades, la pantalla calcula dinámicamente:
 - El porcentaje acumulado y el porcentaje restante (hasta llegar al 100%).
 - La nota final actual de la materia (sumatoria de $(\text{nota} * \text{porcentaje}) / 100$).
 - El estado de aprobación ("Aprobado" si la nota es ≥ 6.0 , "Reprobado" en caso contrario).
- El sistema incluye alertas visuales para notificar al usuario si la sumatoria de porcentajes no ha alcanzado el 100%.

Decisiones de diseño

La arquitectura y las herramientas seleccionadas reflejan un enfoque práctico y estructurado, ideal para aplicaciones locales en Android:

UI Declarativa (Jetpack Compose)

- Se utiliza Compose para construir todas las pantallas (Screens). Se aplican los principios de Material Design 3 (Scaffold, TopAppBar, Card, AlertDialog) para ofrecer una interfaz moderna y coherente.

Gestión de Estado Local

- En lugar de utilizar arquitecturas complejas como MVVM o MVI (que requerirían ViewModels), el estado se maneja directamente en los composables mediante `remember` y `mutableStateOf`. Esto simplifica la base de código para una aplicación de esta escala, aunque acopla un poco la lógica de negocio a la vista.

SQLite Nativo (Sin ORM)

- Se optó por usar la API nativa de Android en lugar de librerías como Room.
- Ventaja: Control absoluto sobre las consultas SQL y cero dependencias de terceros.
- Diseño estructurado: Se creó el objeto `DbStruct` para centralizar los nombres de las tablas y columnas como constantes (`const val`), evitando errores tipográficos ("magic strings") en las consultas.

Patrón Repositorio

- Se implementó correctamente el patrón repositorio (`UserRepository`, `SubjectRepository`, `ActivityRepository`). Esto abstrae la complejidad de la base de datos de las pantallas de Compose, delegando las operaciones CRUD a estas clases.

Conclusiones

Carlos David Guevara Martinez

Jetpack Compose nos pareció una forma más sencilla de crear interfaces en Android, especialmente porque no hay que trabajar con XML como antes. Al principio cuesta un poco entender cómo funciona, pero una vez se comprende, hace que el desarrollo sea más rápido y ordenado. Para quienes están empezando en Android, resulta más práctico de lo que esperábamos.

Fernando Josue Torres Lara

Usar Jetpack Compose facilita bastante el trabajo al momento de diseñar pantallas, ya que todo se hace directamente con código en Kotlin. Esto ayuda a tener menos archivos y a entender mejor cómo se conecta todo. Aunque estamos aprendiendo Android, se siente como una herramienta más clara que las formas tradicionales.

Cristian Omar Pineda Campos

Una de las cosas que más notamos es que la interfaz se actualiza automáticamente cuando cambian los datos, lo que evita tener que hacer muchas cosas manualmente. Esto reduce errores y hace que el código sea más limpio. Para nosotros, que somos nuevos en Android Studio, esto hace que el proceso sea menos complicado.

Geovany Arturo Pineda Fuentes

En general, Jetpack Compose hace que desarrollar aplicaciones se sienta más moderno y organizado. Permite dividir la interfaz en partes pequeñas que se pueden reutilizar, lo que ayuda mucho cuando el proyecto crece. Aunque todavía estamos aprendiendo, se nota que es una buena opción para empezar en el desarrollo Android.

Bibliografía

Android Developers. (s. f.). *Aspectos básicos de Jetpack Compose*.

<https://developer.android.com/codelabs/jetpack-compose-basics?hl=es-419#0>

J. G. Gomila. (2025, octubre 12). *Jetpack Compose en 2025: diseño reactivo y accesible para aplicaciones Android*. Frogames.

<https://cursos.frogamesformacion.com/pages/blog/2025-012-jetpack-compose>

Evidencias

Enlace a github:

https://github.com/MrUnknown023/FORO1_DSM

Enlace a vídeo:

[DSM_FORO1.mp4](#)